Práctica Guiada: Documentación, Trazabilidad y Optimización de Bases de Datos

Nombre del estudiante: Gabriel Antonio Rojas Uriarte

Objetivo: Aplicar conceptos de documentación, trazabilidad, optimización y nodos distribuidos en una base de datos relacional.

Parte 1: Documentación de la Base de Datos

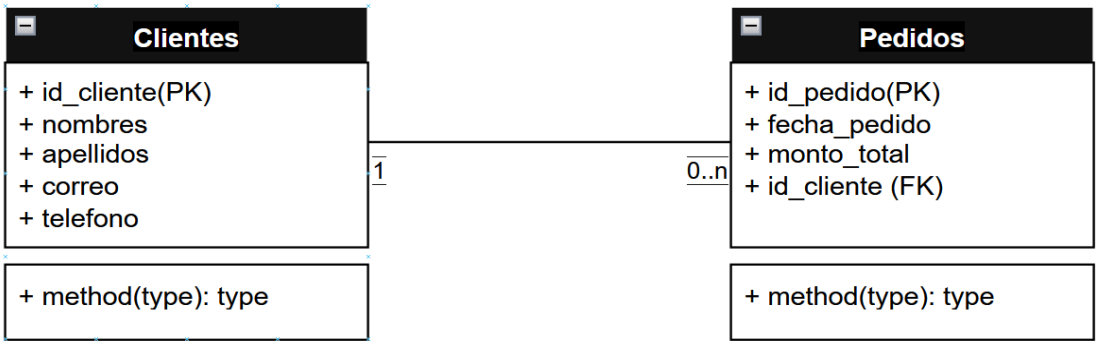
Paso 1: Crear una tabla llamada clientes con los siguientes campos:

- id_cliente (PK)
- nombres
- apellidos
- correo
- telefono

Paso 2: Crear una tabla llamada pedidos con los siguientes campos:

- id_pedido (PK)
- fecha_pedido
- monto_total
- id_cliente (FK)

Paso 3: Dibujar la relación entre ambas tablas e indicar la cardinalidad.



Pregunta: ¿Qué tipo de relación existe entre clientes y pedidos?

Respuesta: La relación entre clientes y pedidos es de uno a muchos, por que un cliente puede tener cero, uno o varios pedidos registrados, mientras que cada pedido solo puede estar asociado a un cliente mediante la clave foránea “id_cliente”.

Parte 2: Diccionario de Datos

Tabla	Campo	Tipo de dato	Descripción
clientes	id_cliente	INT	Identificador único del cliente
clientes	nombres	VARCHAR(100)	Nombre del cliente
pedidos	id_pedido	INT	Identificador del pedido
pedidos	id_cliente	INT	Cliente que realizó el pedido

Parte 3: Trazabilidad de Datos

Paso 1: Crear una tabla auditoria_pedidos con los siguientes campos:

- id_auditoria
- id_pedidoe

- acción
- fecha_cambio
- usuario
-

```sql
-- Crear tabla auditoría
CREATE TABLE auditoria_pedidos (
    id_auditoria INT IDENTITY(1,1),
    id_pedido INT NOT NULL,
    accion VARCHAR(50) NOT NULL,
    fecha_cambio DATETIME NOT NULL,
    usuario VARCHAR(100) NOT NULL,

    CONSTRAINT pk_auditoria_pedidos PRIMARY KEY (id_auditoria),
    CONSTRAINT fk_auditoria_pedidos FOREIGN KEY (id_pedido)
        REFERENCES pedidos(id_pedido)
);
GO
```

Paso 2: Registrar manualmente un ejemplo de cambio realizado sobre un pedido.

Ejemplo: Pedido 101 actualizado por usuario admin el 20/04/2026.

```sql
-- Registrar manualmente un ejemplo de trazabilidad
INSERT INTO auditoria_pedidos (id_pedido, accion, fecha_cambio, usuario)
VALUES
(101, 'Pedido actualizado', '2026-04-20', 'admin');
GO
```

En esta parte se creó la tabla “auditoria_pedidos”, la cual permite registrar cambios realizados sobre los pedidos. Esta tabla almacena el identificador del pedido, la acción realizada, la fecha del cambio y el usuario que hizo la modificación. Luego se registró manualmente un ejemplo indicando que el pedido 101 fue actualizado por el usuario admin el 20/04.2026.

Pregunta: ¿Por qué es importante guardar el historial de cambios?

Guardar el historial de cambios es importante porque permite conocer qué modificantes se realizaron, cuándo ocurrieron y qué usuario las hizo. Esto ayuda a mantener control,

seguridad y trazabilidad dentro de la base de datos, además de facilitar la revisión de errores o cambios no autorizados.

Parte 4: Optimización de la Base de Datos

Paso 1: Identificar qué campos deberían tener índices.

En la base de datos, los campos que deberían tener índices son principalmente las claves primarias y las claves foráneas. En este caso, id_cliente en la tabla Clientes y id_pedido en la tabla Pedidos ya funcionan como identificadores principales. Además, el campo id_cliente en la tabla pedidos debería tener un índice, porque se utiliza para relacionar los pedidos con los clientes y para realizar búsquedas de pedidos pertenecientes a un cliente específico. También podría considerarse un índice en fecha_pedido, ya que puede ser útil para consultas por fecha.

```
-- Crear índice para optimización
CREATE INDEX idx_pedidos_cliente
ON pedidos(id_cliente);
GO
```

Paso 2: Explicar por qué id_cliente en la tabla pedidos debería tener índice.

El campo id_cliente en la tabla pedidos debería tener índice porque se utiliza para relacionar cada pedido con un cliente. Además, este campo se usa en consultas donde se buscan todos los pedidos realizados por un cliente específico. Al tener un índice, SQL Server puede encontrar esos registros más rápido sin revisar toda la tabla, mejorando el rendimiento de las consultas.

Paso 3: Escribir una consulta para buscar todos los pedidos de un cliente específico.

Consulta ejemplo:

SELECT * FROM pedidos WHERE id_cliente = 1;

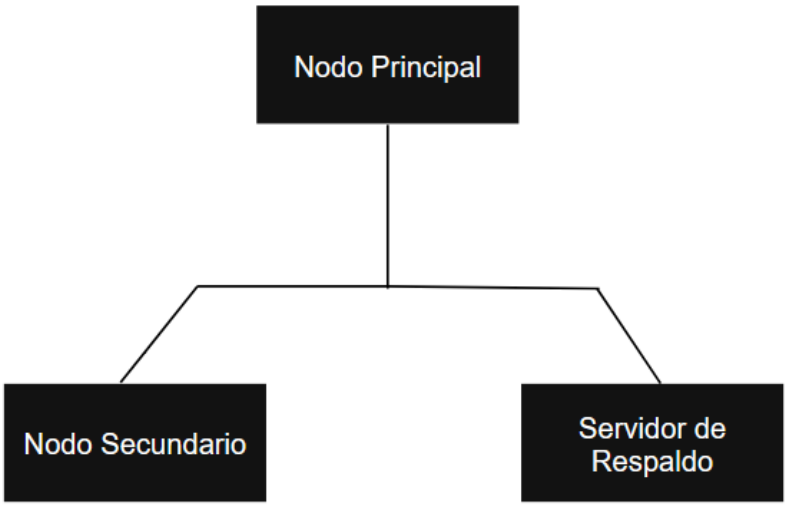
```sql
-- Buscar todos los pedidos de un cliente específico
SELECT *
FROM pedidos
WHERE id_cliente = 1;
GO
```

Se realizó una consulta sobre la tabla pedidos utilizando el campo id_cliente como filto. Esta consulta permite obtener los pedidos asociados a un cliente determinado. En este caso, se buscaron los pedidos del cliente con id_cliente = 1;

Parte 5: Nodos Distribuidos

Paso 1: Dibujar un esquema simple con:

- Nodo principal
- Nodo secundario
- Servidor de respaldo



-

Paso 2: Explicar qué función cumple cada nodo.

Nodo principal: Es el servidor principal donde está la base de datos y donde se ejecutan las operaciones principales como las consultas, inserciones y actualizaciones.

Nodo secundario: Es un servidor de apoyo, puede ayudar a procesar consultas o servir como respaldo en caso de que el nodo principal falle.

Servidor de respaldo: Es donde se guardan las copias de seguridad. Se función es recuperar la información si ocurre un fallo.

Paso 3: Indicar qué ocurriría si el nodo principal falla.

Si el nodo principal falla, el sistema quedaría temporalmente fuera de servicio, ya que las operaciones principales dependen de ese nodo. Sin embargo, el nodo secundario puede asumir parte de las funciones para mantener la disponibilidad del sistema, además, el servidor de respaldo permite recuperar la información mediante copias de seguridad, reduciendo el riesgo de pérdidas de datos.

Actividad Final

Responder las siguientes preguntas:

1. ¿Qué ventajas tiene documentar una base de datos?
   Documentar una base de datos permite comprender mejor su estructura, sus tablas, campos, relaciones y restricciones. También facilita el mantenimiento, la corrección de errores y el trabajo en equipo, porque cualquier persona puede entender cómo está organizada la base de datos.
2. ¿Qué beneficios ofrece la trazabilidad?
   La trazabilidad permite llevar un historial de los cambios realizados en la base de datos. Esto ayuda a saber qué se modificó, cuándo se hizo y qué usuario realizó la acción, lo cual mejora el control, la seguridad y la revisión de errores.
3. ¿Cómo ayuda un índice al rendimiento?
   Un índice ayuda al rendimiento porque permite encontrar datos más rápido sin revisar toda la tabla. Por ejemplo, un índice en id_cliente dentro de la tabla pedidos facilita buscar los pedidos de un cliente específico de forma más eficiente.
4. ¿Por qué es importante documentar nodos distribuidos?

Documentar nodos distribuidos es importante porque permite conocer la función de cada servidor dentro del sistema, como el nodo principal, el nodo secundario y el servidor de respaldo. Esto ayuda a entender cómo se organiza la infraestructura y qué hacer si un nodo falla.